

Кэш Side- Channel атаки и архитектурные методы защиты

Косогова Мария

Группа: Р3206



Введение: кэш Side-Channel атаки

- ❗ Кэш side-channel атаки позволяют извлекать чувствительные данные без прямого доступа к памяти, что делает их особенно опасными для современных многопользовательских и облачных систем.

Права доступа и виртуальная память больше не гарантируют безопасность.

Причина уязвимости:
Оптимизация микроархитектуры процессора создаёт побочные каналы.

Принцип атаки:
Измерение разницы во времени доступа

Зона риска: Облачные вычисления и виртуализация

Кэш и микроархитектура процессора

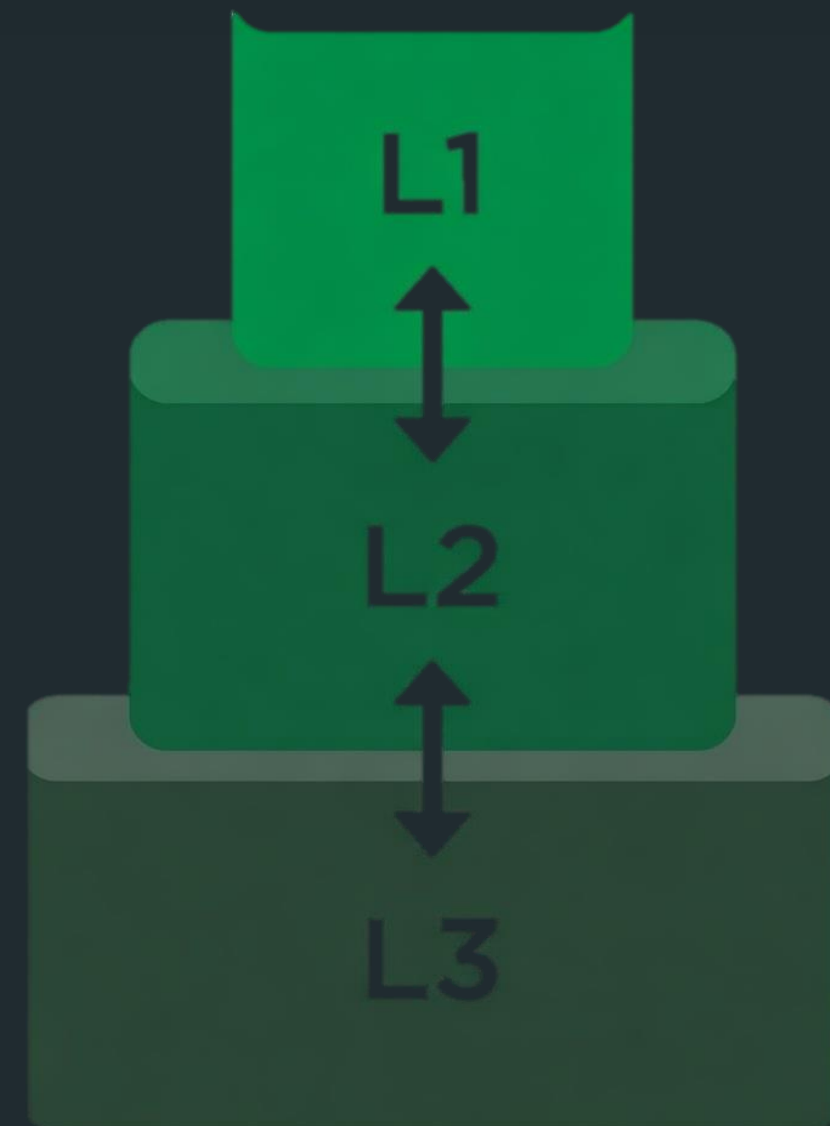
Кэш-память — быстрая память внутри CPU

→ хранит часто используемые данные

Время доступа к памяти:

- L1 — ~4 такта,
- L2 — ~12 тактов,
- L3 — 30–50 тактов,
- DRAM — сотни тактов.

⚠ → разница = основа timing-атак



Модель Side-Channel атаки

МЕХАНИЗМ SIDE-CHANNEL АТАКИ

- 1 — **Контроль состояния кэша**
- 2 — **Выполнение кода жертвы**
- 3 — **Измерение времени доступа**

ⓘ Атака строится на различии между cache hit и cache miss



Flush+Reload работает при наличии общей памяти, например shared libraries



Выбор строки памяти

2

Удаление выбранной строки из кэша (FLUSH)

3

Ожидание запроса строки памяти от жертвы (WAIT)

4

Перезагрузка строки памяти и измерение времени, необходимого для загрузки (RELOAD)

5

Анализ результатов

- быстро → cache hit → жертва обращалась
- медленно → cache miss → не обращалась

6

Повторение процесса (удаление, затем загрузка) для отслеживания того, какой код выполняет процесс жертвы с течением времени.

Prime+Probe

Prime+Probe: A Practical L3 Cache Side-Channel Attack

Этапы Prime+Probe атаки

Prime+Probe не требует общей памяти, использует общий L3

● Prime — заполняем набор своими линиями L3-кэша

● Victim — жертва выполняет операции

● Probe — измеряем время доступа к своим линиям

Спекулятивное исполнение: что это?



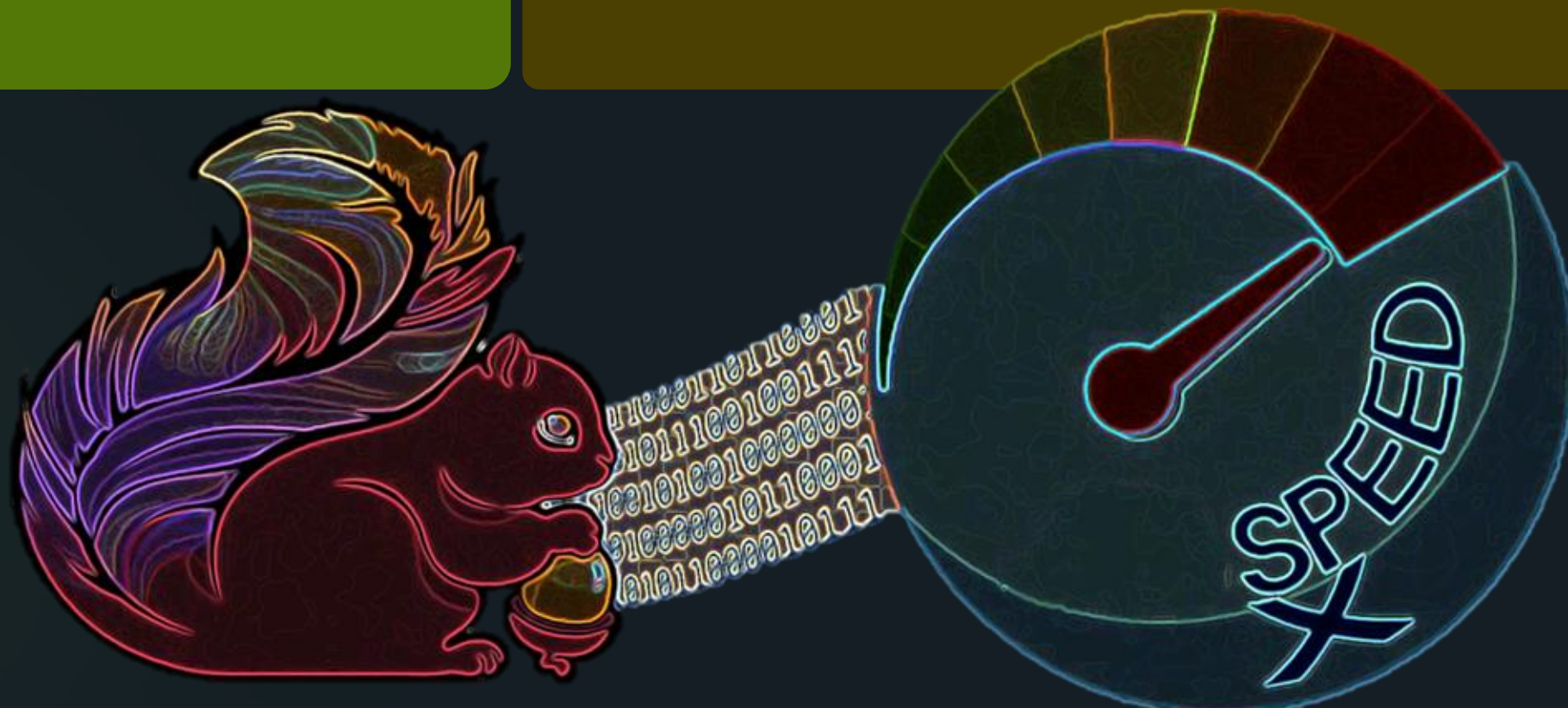
- Оптимизация процессоров для повышения производительности
- Процессор предсказывает условные переходы (branch prediction)
- Исполняет инструкции до проверки условия



Проблема

Микроархитектурные изменения не всегда откатываются: кэш, TLB, предсказатель переходов

- Возникает разрыв между архитектурной моделью и реальным поведением
- Возможность утечки данных через кэш и микроархитектурные каналы



Spectre как комбинация атак

❗ Принцип действия

Branch Predictor + Спекулятивное исполнение + Кэш-канал

Механизм утечки

Спекулятивно
читается секретный
байт



Значение влияет на
микроархитектурное
состояние (кэш)



Атакующий измеряет
через Flush+Reload /
Prime+Probe



Получение секрета по
байту за итерацию

Аппаратные методы защиты

Кэш

Intel CAT — логическое разделение L3-кэша между приложениями

Branch predictor

IBRS — ограничивает привилегированное влияние

STIBP — защищает от влияния SMT-потоков

BPB — очищает состояние предсказателя



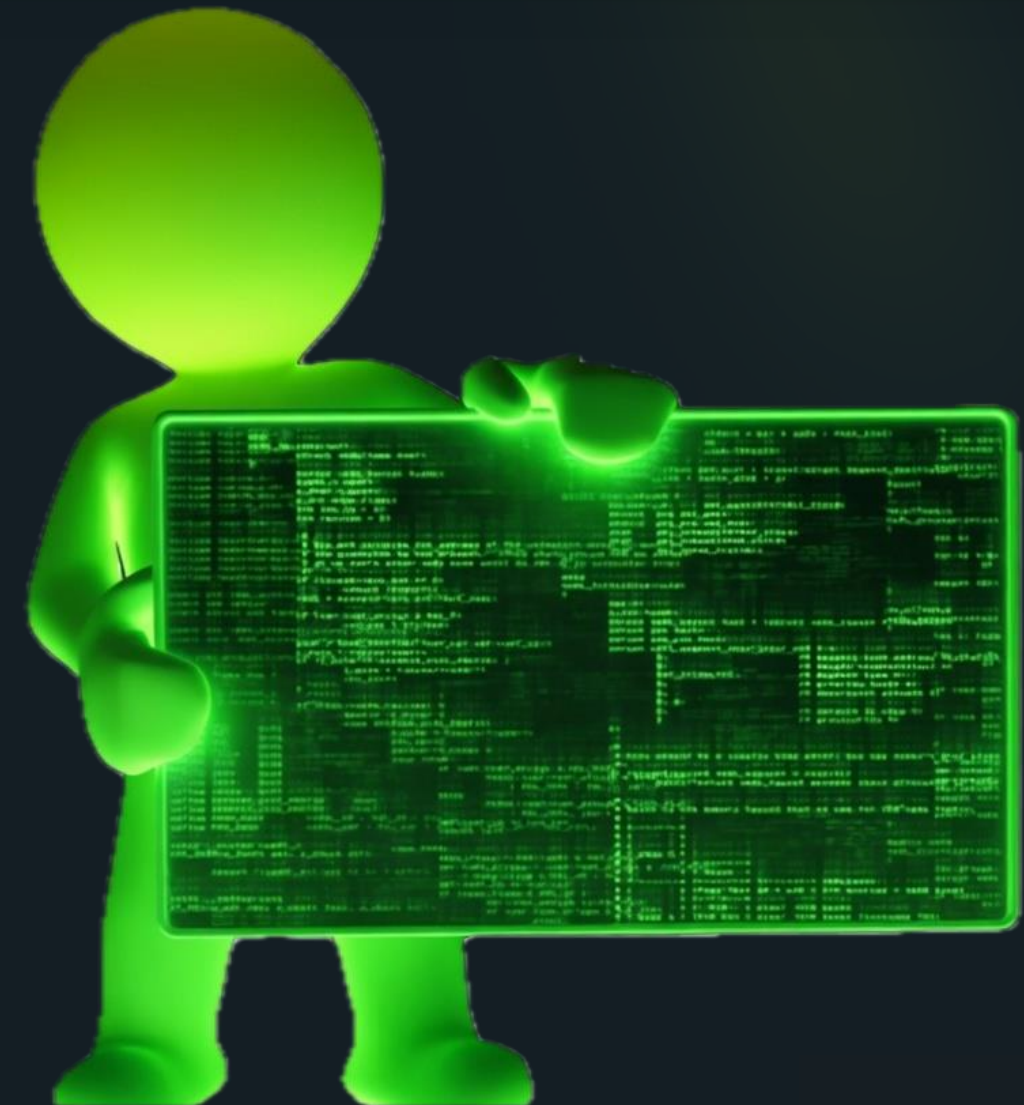
Программные методы защиты

Retpoline - замена косвенных переходов
через возврат из функции

LFENCE — барьер для спекулятивного
исполнения

Constant-time алгоритмы — защита от
кэш-утечек

Отключение SMT — для критичных сред



Итоговый анализ и выводы

Микроархитектура —
полноценная поверхность атаки.

Оптимизации
производительности создают
каналы утечки.

Spectre показал системный
характер проблемы.

Методы защиты: уменьшают риск, но снижают производительность на 5–20% в зависимости от нагрузки.

 Современные серверные ОС включают mitigations по умолчанию, что особенно важно для облачных провайдеров.

Главный вывод: безопасность должна учитываться при проектировании микроархитектуры, а не только на уровне программного обеспечения.

Спасибо за внимание!

СПАСИБО ЗА ВНИМАНИЕ!